Understand the code. See what changes.

C++ and object-oriented programming, explained through memory, behavior and worked examples.

How to use this book Read the diagram, trace the code, then try the pause-and-predict check. Examples use C++17 unless stated. A labeled fragment belongs in its shown context; independent programs are not one combined file.

Instructor exam focus: "array is a constant pointer". The intended cue is a fixed array name with mutable elements. Precisely, an array has its own type and often converts to a pointer to its first element. Know both the cue and the exact C++ rule.

Find a topic

01 Expressions, types and conversions 2
02 Conditions, branches and loops 4
03 Arrays, C strings and records 7
04 Functions, scope and storage 10
05 Declarations, files and the build pipeline 12
06 Pointers and array boundaries 15
07 References and parameter passing 16
08 Callbacks and dynamic allocation 19
09 Recursion and the call stack 22
10 Objects, encapsulation and abstraction 24
11 Constructors, destructors and lifetime 26
12 Inheritance and virtual dispatch 29
13 Copying and ownership 32
14 Overloading, overriding and Complex arithmetic 34
15 UML and object relationships 36
16 Supplementary: pointer chains, const and void* 39
17 Supplementary: instance and static members 41

17 topics · detailed explanations · visual traces · answers included

01 **Foundations**

Expressions, types and conversions

What did this line change?

Types, expression results and stored state

Trace the state

STATEMENT	EXPRESSION RESULT	STORED x
int x = 2;	initial value	2
first = x++;	first gets 2	3
next = ++x;	next gets 4	4

Keep the expression result separate from the value stored after the statement.

PAUSE & PREDICT

A double receives 5/2. What is stored?

A. 2.5 B. 2.0 C. A compilation error

B. Both operands are int, so division produces 2 before conversion to double.

Ask **what did this line do in memory?** Identify whether the expression names a value, an address, a new object, or an alias. Then separate the value produced by an expression from its side effects.

`int x=2; int a=x++ + 3; int b=++x + 3;` produces a=5, b=7, x=4. Postfix supplies the previous value while incrementing x; prefix increments and supplies the updated value. Work through separate full expressions. Do not extend this reasoning to a line that changes the same scalar multiple times without knowing the sequencing rules.

C++ permits procedural, object-oriented and other styles. It is often described as a hybrid language, with substantial C compatibility. That does not mean every C program is a valid C++ program. Ordinary hosted programs use `int main()`, and reaching its end returns zero. A compiler extension accepting `void main` is not a portable language rule.

Variables have types, and types determine permitted operations and representation. An integer and a floating-point value can occupy the same number of bytes while encoding different values. The demonstrated int/float=4, short=2, double=8 and pointer=8 sizes are common environment-specific examples. Use `sizeof`, or an explicitly given platform assumption, in portable reasoning.

Use the type of the operation, not the destination

Integer / truncates toward zero for representable quotients; % supplies the associated remainder. For positive examples, 7/3 is 2 and 7%3 is 1. Multiplication/division/remainder group more tightly than addition/subtraction. Relational operators such as < and equality operators such as == produce a boolean result; assignment = stores a value and is not an equality test.

`double result=5/2;` stores 2.0 because integer division finishes before assignment. `static_cast<double>(5)/2` computes 2.5. Converting a representable finite floating value to int truncates toward zero, so int(-4.8) is -4, not -5. Guard integer denominators against zero rather than memorizing an observed error message.

With int x=2, x+=3 changes x to 5. Evaluating x+3 without storing its result does not. A float literal can use an f suffix, such as 2.5f; a decimal floating literal without it is normally double. A floating value may print 2 rather than 2.0; textual formatting does not establish its type.

`#include <iostream>` makes stream declarations available; std::cout / std::cin name standard streams. << inserts output; >> extracts formatted input. std::endl outputs a newline and flushes; '\n' supplies a newline without inherently requesting that flush. A using-directive is a name-lookup convenience, not an instruction to load every library at runtime.

Declaration is not assignment

`int score;` declares an object without supplying an initial int value. `int score=0;` declares and initializes it. A later `score=8;` assigns to the existing object. Writing `score+3;` evaluates a result and discards it; writing `score+=3;` stores the change. C++ is case-sensitive: score and Score are different names, and `int` is not `Int`. For simple portable identifiers, begin with a letter and use letters, digits and underscores; do not use a reserved keyword. Some underscore patterns are reserved to the implementation, so ordinary descriptive names are safer. Modern C++ can support Unicode identifiers; an ASCII-only classroom convention is not a universal language restriction.

Follow one changing value through successive lines

C++ / WORKED FRAGMENT

```cpp
int x=2;
int first=x++;   // first=2; x becomes 3
int next=++x;    // x becomes 4; next=4
int last=x+5;    // last=9; x remains 4
```

The next line begins with the state left by the previous line. Do not restart x at 2 for every expression. A trace has two lanes: the result supplied to the expression, and the stored state after the statement.

Convert before the operation that needs precision

With `float f=4.5f; int n=4;`, f/n is approximately 1.125. Assigning that result to an int converts afterward and stores 1. In contrast, `int(f)/n` first produces 4, then performs integer division and also produces 1, but by a different route. `double(21/5)` is 4.0; `double(21)/5` is 4.2. A wider destination cannot restore a fraction already discarded.

02 **Foundations**

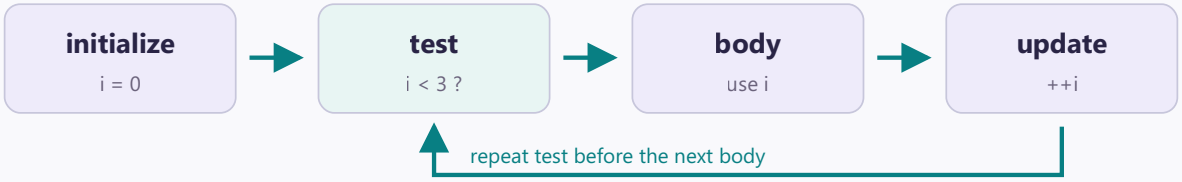
Conditions, branches and loops

Which statements actually run?

Truth, fallthrough, iteration and bit patterns

Follow the loop

FOR LOOP: ONE SETUP, THEN A CYCLE



body values: 0 → 1 → 2

test values: 0 → 1 → 2 → 3

3 bodies; 4 condition checks. A false test leaves the loop.

The final failed condition check matters when counting execution.

PAUSE & PREDICT

The loop for(int i=0;i<3;++i) runs normally. How many condition checks?

A. 2 B. 3 C. 4

C. Three successful tests and the final failed test. The body runs only three times.

Numeric zero is false; every nonzero value is true, including negative values. Built-in `&&` requires both operands true; `||` needs one; `!` reverses truth. `!` has higher precedence than `&&`, which has higher precedence than `||`. Parentheses show grouping clearly. Built-in logical operations short-circuit: the second operand is evaluated only if needed. Distinguish grouping from execution order.

Bitwise `&`, `|`, `~`, `<<`, `>>` act on integer representations. With positive integer examples, `3&4` is 0 and `3|4` is 7, while `3&&4` is true. To test bit k, mask with `1u<<k`. A drawing that flips eight boxes must state that it models eight bits; `~` on an integer is subject to promotions and the type's full width. For example, 18 -> 9 -> 4 is right shifting.

`if/else` selects according to arbitrary boolean conditions. A standard `switch` selects using an integral or enumeration value and distinct constant case values. It jumps into the selected case and then follows ordinary statement flow. Without break/return/another transfer, execution falls through. Two labels may intentionally share one body. `default` is optional, and a break at the very end may be redundant. Arithmetic in a constant case is legal: duplicate converted values are the problem. A switch is not guaranteed faster than equivalent if/else.

`condition ? yes : no` is an expression that evaluates one selected operand. The result can initialize a variable. `x%2==0 ? "even" : "odd"` tests parity; the zero remainder makes the comparison true, rather than making the entire condition false.

A for loop initializes once, then cycles condition → body → update. A while loop checks before entry; a do/while body runs before its first check and therefore runs at least once if reached normally. The for-known-count/while-unknown-count advice is a style convention, not a restriction. Trace the condition each time, including the last failed check. A missing update can cause nontermination, but a loop can also progress by reading input or changing another state. Ordinary looping does not inherently create recursive call frames.

An enumeration gives a type a set of named values. Switch is useful for enum-driven choices. For example, `enum class Action{Attack,Refill};` can be switched with `case Action::Attack:` and `case Action::Refill:`. A scoped enumeration is not an arbitrary string; use its qualified names. Default can handle an unmatched value but is not required for valid switch syntax.

Truth, grouping and short-circuit work

For AND, only true/true gives true. For OR, only false/false gives false. NOT maps false to true and true to false. `!7` is 0 because 7 first counts as true. `!!7` is 1. The numeric value -3 also counts as true.

C++ / WORKED FRAGMENT

```cpp
int visits=0;
bool a=false && (++visits>0); // a=false; visits=0
bool b=true  || (++visits>0); // b=true;  visits=0
bool c=!false || (false && true); // true
```

In c, parentheses show the AND group, but the true left side of OR means that group is skipped. Precedence determines grouping; short-circuiting determines whether the right operand is needed.

Switch labels are entry points

C++ / WORKED FRAGMENT

```cpp
int n=2;
switch(n) {
    case 2:
    case 4: std::cout<<"even ";
    default: std::cout<<"done";
}
// even done
```

Both 2 and 4 enter the same even body. There is no break afterward, so default also runs; default is not a fresh test that rejects a previously matched value. With n=7, only done prints. Removing default would make an unmatched selector execute no case body. A case expression `4/2` duplicates case 2 and is invalid. Distinct character constants such as 'A' can be integral case labels; they are not C++ string labels.

Count bodies and checks separately

C++ / WORKED FRAGMENT

```cpp
int i=3;
do {
    std::cout<<i;
    ++i;
} while(i<5);
// bodies: 3,4; checks: 4<5 true, 5<5 false
```

For `for(int i=0;i<3;++i)`, the body runs three times and the condition is checked four times. Two counters in `for(int i=0,j=2;i<3;++i,--j)` visit (0,2), (1,1), (2,0), not nine pairs. `for(;;)` has an omitted condition treated as true; it still may terminate through break or return. A missing update does not automatically mean stack overflow: a loop ordinarily reuses its frame. Signed counter overflow is undefined behavior, so do not describe an endlessly increasing signed int as a well-defined infinite sequence.

Bit operations need bit lanes

Read bits in aligned lanes

ALIGN THE BIT POSITIONS FIRST

3	0	0	0	0	0	0	1	1
4	0	0	0	0	0	1	0	0
3 & 4	0	0	0	0	0	0	0	0
3 | 4	0	0	0	0	0	1	1	1

Logical 3 && 4 is true. Bitwise 3 & 4 is zero.

An eight-bit display is a model; the actual operation follows the operand type's full width.

The eight-bit display of 3 is 00000011; 4 is 00000100. AND keeps only shared 1 positions, giving 00000000. OR keeps positions set in either lane, giving 00000111, or 7. A right shift removes low-order positions: `3u>>2` is 0, `7u>>2` is 1, `18u>>1` is 9, and `18u>>2` is 4. Use unsigned examples for these introductory bit traces.

`(~7u)&255u` shows an explicitly masked low eight-bit complement, 248. It is different from logical `!7`, which is 0. To test a flag, `flags & (1u<<k)` is nonzero exactly when that bit is set, provided k is a valid bit position. Input buttons and status flags are useful game examples.

Loop transfers: break and continue

`break` exits the nearest enclosing loop or switch. `continue` skips the rest of the current loop iteration. In a for loop, it then reaches the update expression before rechecking; in a while loop, an update placed after continue is skipped, which can prevent progress. A nested loop's break normally leaves only that inner loop. Goto transfers to a label in the function and is not a normal replacement for structured loops; do not infer new function calls from it.

03 Foundations

Arrays, C strings and records

Index, field, or string ending?

Contiguous elements, terminated text and structured records

See the storage



The boundary is not another element. A string terminator is stored, even when invisible.

PAUSE & PREDICT

How much char storage does char name[]="Ada" use?

A. 3 chars B. 4 chars C. A pointer only

B. The three visible letters are followed by the stored null terminator.

One array, many elements of the same type

`int damage[4]={10,20,30,40};` creates four int elements, in order, in one array object. Its bound is fixed for that object. With the classroom assumption sizeof(int)=4, it occupies 16 bytes for its elements. The first element has index zero because it is zero elements from the start. Index three means move three elements, not three bytes.

Use a loop with `i<4`, not `i<=4`. Standard built-in indexing does not check the bound for you. An out-of-range access has undefined behavior even if the next bytes appear readable. Partial aggregate initialization fills remaining elements with zero: `int a[4]={7};` holds 7,0,0,0. `int a[4];` as an ordinary uninitialized automatic local does not give that guarantee.

C++ / WORKED FRAGMENT

```cpp
int damage[4] = {10, 20, 30, 40};
int total = 0;
for (int i = 0; i < 4; ++i) {
    total += damage[i];
}
// total is 100
```

An array parameter `void read(int a[], int count)` is adjusted to a pointer parameter. The function needs count; `sizeof(a)` there measures a pointer. At the caller, `sizeof(damage)/sizeof(damage[0])` yields the element count because damage is still an array expression in sizeof. References to arrays are an alternative beyond the basic course pattern.

Rows are arrays too

Choose a row, then a column

A TWO-DIMENSIONAL ARRAY IS AN ARRAY OF ROW ARRAYS



grid[1][2] is 6. Two rows × three columns = six visits.

The inner loop restarts for each row. Preserve each row's array boundary.

`int grid[2][3]={{1,2,3},{4,5,6}};` is an array of two row arrays, each with three ints. `grid[1][2]` is 6. Row-major ordering means the conceptual sequence is 1,2,3,4,5,6. A conceptual flattened position is row*3+column. For portable pointer reasoning, preserve row boundaries: `grid[0]+3` is one-past the first row and must not be dereferenced as if it were that row's fourth element.

Nested loops restart the inner counter for each outer iteration. Two rows and three columns visit six cells. Two counters updated in one for loop are not a nested loop; they do not generate every pair.

C++ / WORKED FRAGMENT

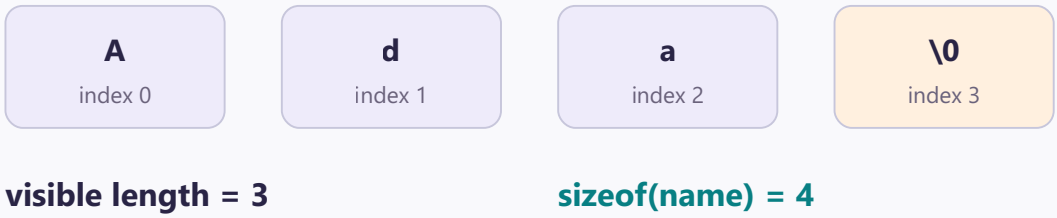
```cpp
int grid[2][3] = {{1,2,3},{4,5,6}};
int total = 0;
for (int row = 0; row < 2; ++row)
    for (int col = 0; col < 3; ++col)
        total += grid[row][col];
// total is 21
```

C strings need an ending

`char name[]="Ada";` contains four chars: A,d,a and '\0'. The terminator occupies storage but is not a visible letter. `sizeof(name)` is 4; its C-string length is 3. `char name[3]={'A','d','a'};` is a valid array but is not a null-terminated C string. Passing it to an operation expecting a C string may read beyond it. In C++, `char name[3]="Ada";` is ill-formed because there is no space for the terminator.

Visible text versus stored characters

THE ENDING OCCUPIES STORAGE



The null terminator ends the text; it is not a visible letter.

`std::string` manages a string value and its storage. It can grow, copy its content and compare values. Formatted `std::cin>>name` for a string reads one whitespace-delimited token; `std::getline(std::cin,name)` can read spaces. After formatted extraction, an unconsumed newline can cause the next getline to return an empty line; consuming that delimiter is a separate input step. Avoid unbounded input into a fixed-size character buffer.

Records contain different types

An array groups homogeneous elements; a struct groups named members that may have different types. The class/struct type tells the compiler each member's role.

C++ / WORKED FRAGMENT

```cpp
struct Employee {
    int id;
    char name[10];
    float salary;
};
Employee team[2] = {{1,"Ada",1000.0f},{2,"Lin",1200.0f}};
// team[1].id is 2; team[0].name is "Ada".
```

Do not calculate sizeof(Employee) as a universal 4+10+4=18. Implementations may insert padding so members meet alignment requirements. A common layout with 4-byte int/float alignment is 4 bytes ID, 10 name, 2 padding, 4 salary =20. That is an example, not a mandated size. Padding is unused layout space, not another member.

For `Employee* p=&team[0]`, use `p->salary`, equivalent to `(*p).salary`. `p+1` advances to the next Employee including its complete size/padding. C++ structs support methods and inheritance; the course's simple-record convention does not restrict the language.

> Memory anchor: **index chooses the element, member name chooses the field, terminator ends the text**.

Keep array traversal and record traversal distinct

`team[i].salary` first chooses an Employee in the array, then its salary field. An array of records is homogeneous at the element level: every element is an Employee, even though each Employee contains mixed member types. `p+1` moves by sizeof(Employee), including padding, while `p->salary` selects one field of the current record. Equal sizes of a small record and a pointer on one machine do not make them the same kind of object.

Why the next input appears to be skipped

After `std::cin>>age`, formatted integer extraction normally leaves the following newline unread. A subsequent get or getline can consume that newline immediately. Calling get twice may appear to fix one particular input sequence, but it is not a general input policy. If a complete following line is wanted, discard the remainder of the current line deliberately:

C++ / WORKED FRAGMENT

```cpp
#include <iostream>
#include <limits>
#include <string>
// Inside main, after a successful std::cin >> age:
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
std::string name;
std::getline(std::cin,name);
```

This preserves leading spaces in the next line. `std::getline(std::cin>>std::ws,name)` is another useful policy when discarding all leading whitespace is intended; it also discards blank lines, so the two policies are not identical.

---

04 **Foundations**

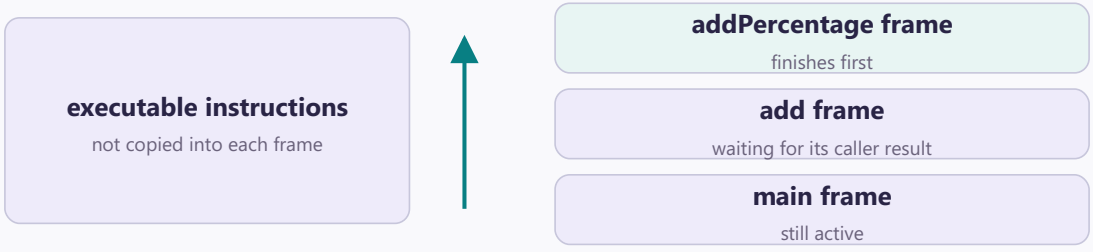
Functions, scope and storage

Local scope. Lasting state.

Declarations, call frames, shadowing and storage duration

Follow the active calls

CODE IS SHARED; INVOCATIONS HAVE THEIR OWN DATA



A call pushes a frame in this model; a return resumes the waiting caller.

This common implementation model separates instructions from call-specific data.

PAUSE & PREDICT

A local static changes from3 to4 on the first call. What happens on the next call?

A. It keeps4 before the next increment B. It must start at3 again C. It becomes a global name

A. The object persists across calls, while the name remains local.

A function packages a named operation. Its declaration makes its name and type available; its definition provides the body. A declaration before use lets main call a definition appearing later. A non-void function must provide a suitable result on paths where it returns, except main's special fallthrough rule. A void function performs work without supplying a value.

The call-stack drawing is a useful model: main calls add; add calls addPercentage; the innermost call finishes first. Each invocation has its own automatic locals. Executable instructions are not copied wholesale into each call frame. Registers and optimizations may change physical details without changing the program's observable meaning.

Scope answers where a name can be used. **Lifetime/ storage duration** answers how long an object exists. A local static keeps its value across calls but its name remains local. For `static int s=3; return ++s;`, successive calls in one run return 4,5,6. An ordinary local initialized to 3 on each invocation returns 4 each time. Restarting the process does not preserve either variable as a saved game.

A common implementation diagram has text/code, static data, stack and heap. Treat it as a common implementation model. Automatic locals normally use stack storage; new-expressions obtain dynamic storage; static/global objects have static storage duration. A local pointer can be automatic while its target is dynamic. A variable without new is not automatically a stack variable: globals and static locals are counterexamples.

Same initialization, different duration

Follow three calls

NAME VISIBILITY ≠ OBJECT DURATION

ordinary local	**call 1: 4**	**call 2: 4**	**call 3: 4**
local static	**call 1: 4**	**call 2: 5**	**call 3: 6**

Both names stay local. Only the static object retains its changed value.

Visibility and duration are different dimensions of the same program.

A function has a contract and a body

In `int sum(int a,int b){return a+b;}`, int is the return type, sum is the name, a/b are parameters, and the braces enclose the definition's body. `int total=sum(2,3);` uses the returned value. A void printSum function could print 5 instead, but the caller would receive no numerical result. Returning data and displaying data are separate responsibilities; a reusable calculation need not decide its output format.

The same spelling can name a different object

C++ / WORKED FRAGMENT

```cpp
int n=5;
{
    int n=8;
    std::cout<<n++; // prints 8; inner n becomes 9
}
std::cout<<n;       // prints 5; outer n was unchanged
```

The inner declaration shadows the outer name; it does not overwrite the outer object. The inner name stops being usable after the closing brace. A global object has a different scope from a local with the same spelling; global does not mean every context ignores declaration order and name hiding. A program-counter register tracks instruction execution in a typical machine model. Call frames hold call-specific data; they are not fresh copies of the function's machine instructions.

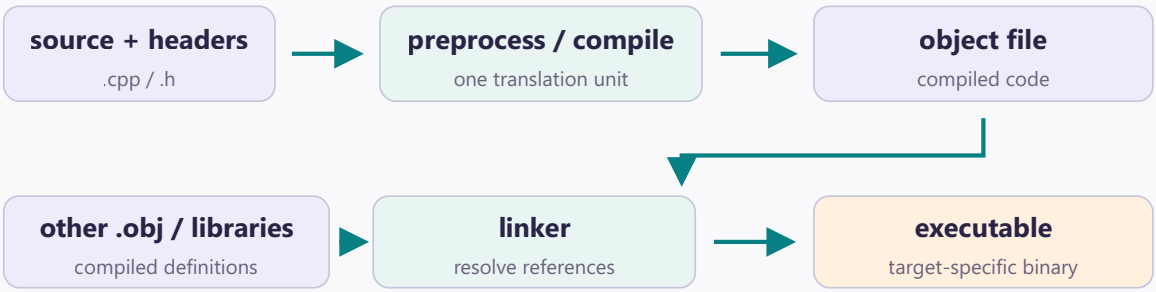
05 **Foundations** Declarations, files and the build pipeline

How does source become a program?

Headers, definitions, compilation and linking

Follow the artifacts

SOURCE TEXT → COMPILED PIECES → RUNNABLE PROGRAM



A header declares the interface; compiled definitions supply the linked behavior.

PAUSE & PREDICT

A function is declared and used, but its required definition is absent. Which build stage commonly reports it?

A. Preprocessing B. Linking C. Runtime output

B. The declaration lets compilation check the call; the linker still needs the compiled definition.

Know a name before using it

A declaration tells the compiler what an entity is. A definition supplies the object or function body as appropriate. `int add(int,int);` declares a function; `int add(int a,int b){return a+b;}` defines it. Parameter names may be omitted in a prototype. A call in main can use a later definition if a suitable declaration is already visible.

C++ / WORKED PROGRAM

```cpp
int add(int, int);  // declaration visible here
int main() {
    return add(2,3) == 5 ? 0 : 1;
}
int add(int a, int b) { return a+b; }
```

A forward declaration `class Derived;` is not sufficient to create a Derived object by value; its complete definition is needed. A function prototype also does not supply its implementation to the linker.

Separate interface from implementation

C++ / WORKED FRAGMENT

```cpp
// Weapon.h
#ifndef COURSE_WEAPON_H
#define COURSE_WEAPON_H
class Weapon {
    int damage;
public:
    explicit Weapon(int value);
    int getDamage() const;
};
#endif
```

C++ / WORKED FRAGMENT

```cpp
// Weapon.cpp
#include "Weapon.h"
Weapon::Weapon(int value) : damage(value) {}
int Weapon::getDamage() const { return damage; }
```

C++ / WORKED PROGRAM

```cpp
// main.cpp
#include "Weapon.h"
int main() {
    Weapon sword(10);
    return sword.getDamage() == 10 ? 0 : 1;
}
```

Compile and link both cpp files as part of the program. Including the header makes declarations visible; it does not automatically compile an unrelated cpp file. A bare `int getDamage() const` outside the class is not the same definition; the Weapon:: qualification ties the member to its class.

Definitions inside a class body are implicitly inline in this ordinary setting. A header-only class can therefore be valid. File separation is organization, but definitions and repeated-definition rules still matter. Include guards prevent the same header's class definition from being inserted twice into one translation unit. Do not solve duplicate definitions by including arbitrary cpp files into every caller.

Follow the build artifacts

Preprocessing handles directives such as #include. Compilation checks/parses the resulting translation unit and produces target code; an assembler stage may be distinct. The linker resolves references across compiled object files and libraries. An object file is an intermediate binary artifact; it is not an OOP object instance like sword. Those two meanings of 'object' are unrelated.

Calling an undeclared add is a compilation problem. Declaring add, calling it, but omitting its required definition commonly produces an unresolved-symbol link error. Duplicating externally linked ordinary function definitions across translation units commonly produces a multiple-definition issue. Exact diagnostics depend on the toolchain; the conceptual stage is what matters.

An executable is built for a target environment. Portable source code can be rebuilt for another target; that does not make the same Windows exe a native executable on every system. A Windows executable, for example, remains a target-specific binary.

Trace code in this order

Check validity before tracing output. First declarations/types/access, then lifetime/bounds, then dispatch, then arithmetic/output. If compilation fails, do not invent a runtime result. If undefined behavior is reached, do not treat one observed run as a portable answer. If the question depends on a machine size, look for the stated assumption.

> Memory anchor: **header tells what exists; source defines behavior; linker joins the compiled pieces**.

A member and a free function are different entities

`void Weapon::attack(){...}` defines the declared Weapon member. `void attack(){...}` defines a free function even if its spelling matches. A non-static member is called on an object, such as `sword.attack()` or `knife->attack()`. Calling `Weapon::attack()` without an object is invalid for that ordinary member. Free functions themselves are valid C++; putting every operation inside a class is not a language requirement.

Use `#include "Weapon.h"` for the project's header and `#include <iostream>` for the standard stream header. Headers are processed as declarations/source text, not executed as runtime function calls. A class can be defined in a cpp file when its uses have the required visibility there; it is not forbidden by its extension. A header makes the same interface conveniently available to multiple translation units. Distributing compiled code plus a header lets callers compile against declarations and link against an implementation without receiving its original source text.

06 Memory

Pointers and array boundaries

An address is a route.

Pointer objects, pointees and array boundaries

Name the route and target

THREE EXPRESSIONS. THREE DIFFERENT MEANINGS.



&p addresses the pointer object

***p** designates x

p = &y moves the route. *p = 105 writes the current target.

Addresses are symbolic routes here, not measured machine addresses.

PAUSE & PREDICT

With int* p=&x, what does &p address?

A. x B. A copy of x C. The pointer object p

C. p is its own object. Its stored value points to x, but &p locates p itself.

For `int x=100; int* p=&x;`, x is an int object and p is a separate pointer object. `&x` obtains x's address. Reading p obtains its stored pointer value. `*p` designates x, so assigning `*p=105` changes x. `&p` obtains the pointer object's own address. Draw the pointer object separately from its target to remember this distinction.

The star in a declaration is part of a pointer declarator; the unary star in an expression dereferences a pointer. In `int* p, q;`, p is a pointer and q is an int. Declare one variable per line when learning. Initialize an absent target with nullptr, then assign a valid address before dereferencing. Null, indeterminate and dangling pointers are different states, and none can be dereferenced safely.

Copying a pointer does not copy its target. `int* q=p;` gives two pointer objects pointing to the same x. `*q=7` changes what both see; `q=&y` changes q's target while p still points to x.

Within `int a[3]={10,20,30}`, a pointer to a[0] can advance by array elements. `p+1` means the next int, not the next byte. `a[i]` is defined in terms of `*(a+i)`. You may form a pointer one past the end, but may not dereference it. Subtraction within the same array gives an element distance; adding two pointers is ill-formed. Arbitrary accesses do not become valid merely because they stay in the process's stack region.

Instructor exam focus: 'array is a constant pointer'. Remember the intended meaning: the array name cannot be reseated, and ordinary uses often convert it to a pointer to its first element. Precisely, the array is an array object. `sizeof(a)` measures the complete array, unlike sizeof of a pointer. `&a` has pointer-to-array type. An array parameter written `int a[]` adjusts to a pointer; its size information is not carried as an array value.

The address stored in p can be copied without copying x. Reassigning p moves the route; assigning through *p changes the target.

A value calculation does not move the arrow

C++ / WORKED FRAGMENT

```cpp
int a[2]={100,200};
int* p=a;
int v=*p+5;      // v=105; p still points to a[0]
int w=*(p+1);    // w=200; p still points to a[0]
++p;             // now p points to a[1]
```

The first line of calculation adds to a value. The second calculates another position without assigning it to p. Only the final increment changes p. For a lone int rather than a two-element array, p+1 may denote one-past, but dereferencing that position is invalid. The fact that some other variable happens to be physically nearby does not provide an array relationship.

Typed arithmetic already scales: a+i selects the element i positions from the beginning. In a byte-address illustration, the offset is i*sizeof(int). Do not multiply by sizeof(int) again inside a+i. A pointer type determines the accessible object interface and lawful operations; matching byte sizes alone does not authorize reading a float as an int through a cast pointer.

07 **Memory** References and parameter passing

Did you copy data or pass a route?

Values, aliases, pointer copies and caller mutation

Compare the three calls

int n	x [5] | n [5]	++n → x stays 5
int* p	p [address of x] → x [5]	++*p → x becomes 6
int& n	x and n → one int [5]	++n → x becomes 6

A copied pointer is separate; its target can still be the caller's original object.

PAUSE & PREDICT

void f(int* p){++*p;} is called as f(&x). Can x change?

A. Yes B. No, because p is copied C. Only if p is static

A. The address is copied into p; dereferencing that copied address still reaches x.

`int& r=x;` binds a reference name to the existing x. There is one int object, with two names. `r=9` writes x. Given a separate y, `r=y` copies y's value into x; it does not make r refer to y. A reference must be initialized to a suitable live object and cannot be reseated. A reference can still outlive its referent and dangle; it is not automatic memory ownership.

When p points to live x, `int& r=*p;` binds r to x as well. It does not create a copy and it does not validate p.

Compare three parameter forms:

Definition	Call	Effect of increment inside
`void f(int n){++n;}`	`f(x)`	changes local copy
`void g(int* p){++*p;}`	`g(&x)`	changes caller's x
`void h(int& n){++n;}`	`h(x)`	changes caller's x

Passing a pointer by value copies the address; it still permits mutation through that address. Assigning a new address to the local pointer does not reseat a caller's pointer. For read-only access to a larger object, a const reference can avoid a value copy while restricting mutation through the parameter.

Be careful with these two expressions: `*p++` parses as `*(p++)`, while `(*p)++` changes the pointed-to object. Use parentheses and draw the arrow separately from the data. A copied pointer can still change its original target through dereference.

Trace the same caller through all three forms

C++ / WORKED PROGRAM

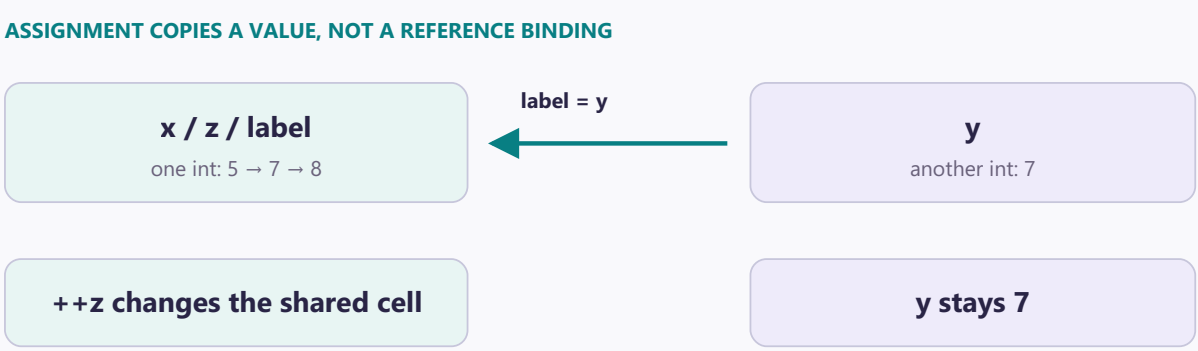
```cpp
#include <iostream>
void byValue(int n) { ++n; }
void byAddress(int* p) { ++(*p); }
void byReference(int& n) { ++n; }
int main() {
    int x=5;
    byValue(x);
    std::cout << x << ","; // 5: separate local value changed
    byAddress(&x);
    std::cout << x << ","; // 6: copied address leads back to x
    byReference(x);
    std::cout << x;        // 7: parameter aliases x
}
```

The second and third operations both affect x, but their parameters are not identical kinds of entity. The pointer parameter is a new pointer object initialized with an address. The reference parameter binds a new name to the existing int. If the second function assigns its local p=nullptr, the caller's x remains alive, and a caller pointer would not automatically be reseated. Mutation through a pointer, mutation of a pointer, and reference binding should be drawn as three different actions.

The value parameter protects only what was copied. Passing a class by value can still share an external resource when that class contains a shallowly copied raw pointer. Therefore 'pass by value always means deep copy' is too broad. The copy lesson explains how the class's own copy semantics determine what lies behind its copied members.

Assignment through an alias does not change the alias

Several labels, one cell



A new value can enter the cell without changing what any reference names.

C++ / WORKED FRAGMENT

```cpp
int x=5, y=7;
int& z=x;
int& label=z;
label=y;  // x, z, label now observe 7; y is separate
++z;      // x, z, label now observe 8; y is still 7
```

Draw one cell with the labels x/z/label, and another cell labeled y. Assignment copies 7 between the cells. It does not move label onto y. Likewise, with a valid `int* p=&x`, `int& r=*p` binds to x. Reassigning p later does not reseat r. Never bind by dereferencing a null or dangling pointer.

To remember parameter initialization, picture the call as three different first steps: `int n=x` for a value parameter, `int* p=&x` for a pointer parameter called with &x, and `int& n=x` for a reference parameter. Only the first makes a new int value. The second makes a new pointer that still reaches the old int.

08 Memory

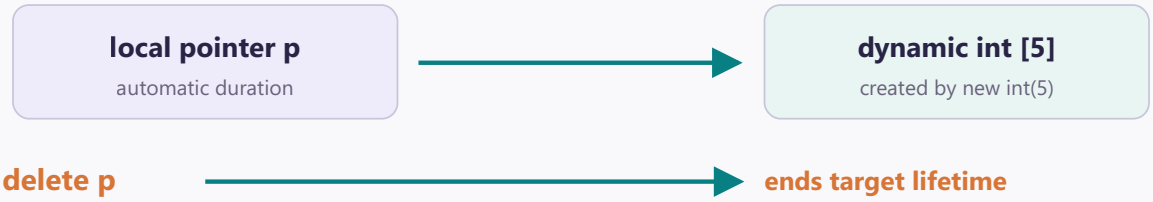
Callbacks and dynamic allocation

A pointer and its target live separately.

Selectable callbacks and matched allocation/cleanup

Separate lifetimes

TWO SEPARATE OBJECTS AND LIFETIMES



delete p → ends target lifetime

p and copied aliases are not automatically cleared by deletion.

Destroying a local pointer does not automatically destroy a dynamic target.

PAUSE & PREDICT

Which release matches new int[3]?

A. delete p B. delete[] p C. Neither is needed

B. The array allocation and array deletion forms must match.

`int (*calculate)(int,int)=add;` declares a pointer to a function returning int and accepting two ints. Parentheses matter: `int* calculate(int,int)` declares a function returning an int pointer. Changing parameter names changes neither compatible function type nor overload identity. Reassign calculate to subtract or multiply with the same type; `calculate(2,3)` invokes the current target. The function name, rather than add(2,3)'s result, supplies the function pointer. This makes behavior selectable at runtime.

`int* p=new int(5);` creates a dynamic int initialized to 5 and stores its address in p. `new int` without an initializer does not zero-initialize that scalar. The dynamic object and p have separate lifetimes. Deleting p releases the target; it does not automatically clear p or any aliases. Losing the only owning address before deletion leaks storage. Match `new T` with delete and `new T[n]` with delete[].

Raw-pointer examples make the memory model visible. In practical owning code, value members and RAII (resource acquisition is initialization) help tie cleanup to object lifetime.

A dynamic array and a dynamic record

The following complete example uses initialized storage and matches each allocation form to its cleanup form.

C++ / WORKED PROGRAM

```cpp
#include <iostream>
struct Employee { int id; float salary; };
int main() {
    int count=3;
    int* data=new int[count]{};  // three zero-initialized ints
    for(int i=0;i<count;++i) data[i]=(i+1)*10;
    for(int i=0;i<count;++i) std::cout<<data[i]<<",";
    Employee* employee=new Employee{7,100.0f};
    std::cout<<employee->id;
    delete[] data;              // array allocation
    delete employee;           // one Employee
}
```

data is a pointer, not an array object with a bound attached to its type. Its dynamic target contains count elements, and the program must remember that count for iteration. new int[count] permits a runtime count; writing a built-in automatic array int data[count] with a nonconstant count is not standard C++17. The latter may be accepted as a compiler extension, which is a separate portability issue.

An allocation can fail. Ordinary throwing new normally reports failure by an exception; the nothrow form instead returns a null pointer on allocation failure. Do not assume ordinary new quietly returns nullptr and that a subsequent if check always handles allocation failure. The central pattern is valid allocation, use, ownership and matched release.

An array of behaviors is different from a function taking an array

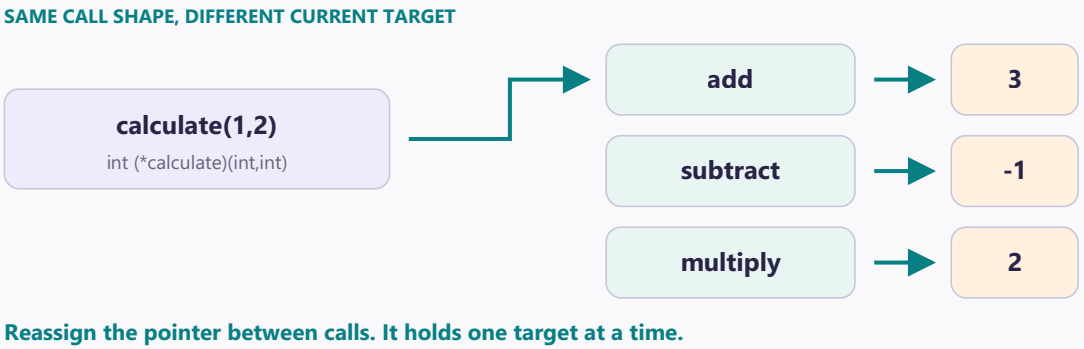
C++ / WORKED PROGRAM

```cpp
#include <iostream>
int add(int a,int b){ return a+b; }
int subtract(int a,int b){ return a-b; }
int multiply(int a,int b){ return a*b; }
int main(){
    int (*operations[3])(int,int)={add,subtract,multiply};
    for(auto operation:operations) std::cout<<operation(2,3)<<",";
}
```

operations is an array whose elements are function pointers. Each element has the same compatible target type; the pointed-to functions need not share parameter names. A menu can select an element, but validate the menu index before indexing. Passing such a function pointer to another function is a callback pattern: the recipient invokes supplied behavior instead of hardcoding one operation.

One current function target

Choose a behavior



The target's return type and parameter types must match the function-pointer type.

Given compatible add/subtract/multiply functions, a pointer calculate can be assigned each in turn. `calculate(1,2)` returns 3, then -1, then 2 after those successive target assignments. One function pointer holds one current target; an array of function pointers holds several selectable targets. This is runtime behavior selection, not virtual overriding, and it does not require copying function bodies into a pointer object.

Equal values do not imply shared storage

C++ / WORKED FRAGMENT

```cpp
float f=5.5f;
float* p=new float(5.5f);
*p=7.0f; // changes the dynamic float; f remains 5.5f
delete p;
p=nullptr;
```

There are two float objects, despite their equal initial values. p is a third object containing an address. `&p` is the pointer object's own address; p is the dynamic target's address; *p designates that target before deletion. Clearing this one pointer after deletion does not clear other aliases. A bare `new Weapon;` can allocate successfully even if its result is discarded; losing the only owning address makes later use and cleanup impossible in that example.

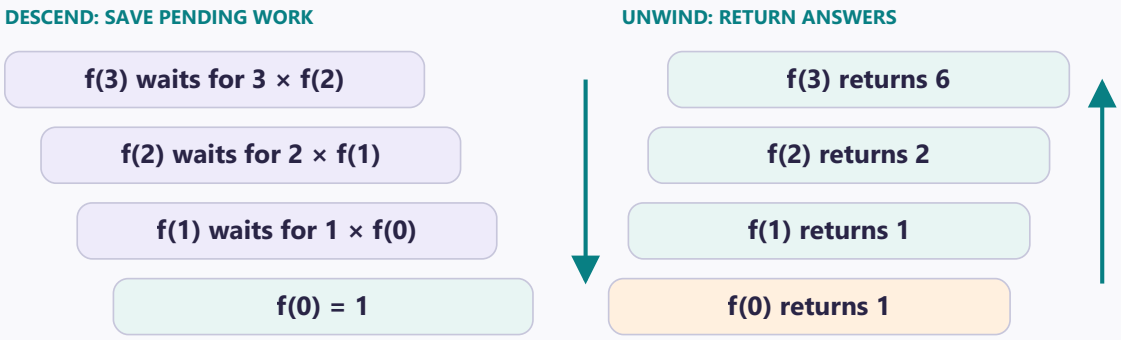
09 Memory

Recursion and the call stack

Go down with work. Come back with answers.

Base cases, progress, pending work and unwinding

Down the stack and back



Read down for calls and up for returned answers. Four factorial calls are active at the deepest point.

PAUSE & PREDICT

With factorial(0) as the base case, how many factorial calls does factorial(3) make, including itself?

A. 3 B. 4 C. 6

B. The calls are f(3), f(2), f(1), f(0). The numerical result is6, a different count.

Recursion solves a problem using smaller instances of that problem. A base case supplies a direct result; a recursive case must progress towards it. For nonnegative n, factorial(0)=1 and factorial(n)=n*factorial(n-1). A version stopping only at one must reject or handle zero separately.

For factorial(4), record pending work: 4*factorial(3), then 3*factorial(2), then 2*factorial(1). Return 1, then 2, then 6, then 24. The multiplications finish during unwinding, after the recursive calls produce their answers. For a base at zero there is one extra call. Distinguish number of calls, maximum depth, and result.

`power(base,exponent)` for a nonnegative integer exponent returns 1 at exponent zero and otherwise base*power(base,exponent-1). Base stays unchanged; exponent measures remaining work. Negative exponents are outside this simple example's input contract. They need a separate specification and usually a non-integer result type. Unbounded recursion can exhaust the stack, and finite recursion can still overflow an integer result.

A complete small-domain example

C++ / WORKED PROGRAM

```cpp
#include <iostream>
// Preconditions for this teaching example: 0 <= n <= 6.
int factorial(int n) {
    if(n==0) return 1;
    return n*factorial(n-1);
}
// Preconditions here: exponent >= 0; chosen result fits int.
int power(int base,int exponent) {
    if(exponent==0) return 1;
    return base*power(base,exponent-1);
}
int main() {
    std::cout<<factorial(4)<<","<<power(2,3); // 24,8
}
```

For factorial(3), the stack reaches four factorial invocations: f(3), f(2), f(1), f(0). The base returns 1. The waiting f(1) computes 1*1; f(2) computes 2*1; f(3) computes 3*2. The same local name n holds a different value in each call. Returning from f(0) does not reset the outer n values or end all calls at once.

Active computation	Still waiting to do	Returned answer
f(3)	3 times the answer from f(2)	6
f(2)	2 times the answer from f(1)	2
f(1)	1 times the answer from f(0)	1
f(0)	no recursive work	1

Read the first column top to bottom for descent, then returned answers bottom to top for unwinding. The return statement contains work after the recursive result arrives, so this factorial form is not tail recursion. A different accumulator version can put the recursive call in tail position, but C++ does not require a compiler to eliminate its call frames.

There are two independent failure questions. First, does the input reach the base? Passing a negative exponent to the shown decrementing recurrence moves farther from zero. Second, even for terminating inputs, does the arithmetic result fit the return type? A recursion can terminate logically yet overflow signed int. Keep teaching inputs small enough that the chosen result type can represent the answer.

An iterative factorial can store an accumulator in one active invocation, while this recursive version keeps one waiting invocation per level. Recursion is useful when the problem's structure is naturally recursive, not because every repeated multiplication needs a fresh frame.

Why exponent decreases but base does not

`power(2,3)` means three copies of 2 multiplied together. Each step removes one remaining factor: `2*power(2,2)`, then `2*power(2,1)`, then `2*power(2,0)`. The base stays 2; the exponent counts remaining work. On return, the answers are 1,2,4,8. Replacing the base with base-1 would solve a different recurrence. A negative exponent with only exponent==0 as the stop and --exponent as progress moves away from zero, so it violates this function's input contract.

The relay analogy works because each caller delegates a smaller instance, then waits for its answer. Recursion needs both a reachable stop and a smaller next problem. Repetition alone is not enough. Stopping factorial at 1 works for positive inputs; supporting 0! requires a base rule that handles zero too. Do not multiply the accumulated result by zero: 0! contributes the identity value 1.

10 OOP

Objects, encapsulation and abstraction

Give an object a useful interface.

Encapsulation, abstraction and per-instance state

State behind an interface



Encapsulation: control change. Abstraction: expose what callers need.

Expose an operation that preserves valid state rather than unrestricted field changes.

PAUSE & PREDICT

Which statement about C++ struct is correct?

A. It cannot inherit B. It always allocates on the stack C. It can have private members and methods

C. Struct supports class features; its default member and base access is public.

A class defines a type; an object is an instance with its own non-static state. Encapsulation groups state with operations and controls access so valid state can be maintained. A private damage member plus a validating public setter can prevent negative damage. A getter need not imply a setter; read access and mutation are separate decisions. Private is a language access rule, not encryption or authentication.

Abstraction exposes the useful model and hides unnecessary detail. Keep functions focused and state relevant: Weapon should carry state needed to represent a weapon. Distinguish this interface-design

idea from encapsulation. Inheritance models an appropriate is-a relationship. Polymorphism provides different behavior through common names/interfaces.

`class` defaults to private member access and private base inheritance; `struct` defaults to public for both. Both can have functions, constructors, access sections, virtual functions and inheritance in C++. Using struct for simple public records and class for invariant-bearing types is a convention.

`Weapon sword;` creates an automatic local object in an ordinary block. `Weapon* p=new Weapon;` creates a dynamic object. Use sword.attack() for the object and p->attack() for the pointer. A class definition does not determine that all its instances live on the heap.

Choose state and permissions deliberately

A Weapon can expose attack while keeping its damage state private. A Human model need not contain a car field merely because some humans drive: include what this program's responsibility requires. Unnecessary per-instance data is multiplied across every instance, though exact object size also depends on layout and alignment. Abstraction keeps the model relevant and the public interface understandable; encapsulation groups state with behavior and controls its permitted changes.

A read-only ID can have a getter without a public setter. A write-only operation is possible too, such as supplying a secret that callers cannot read back; it is not syntactically or conceptually impossible merely because a paired getter is absent. An access specifier applies until the next specifier or the end of the class, to both data and functions. Private access is a compile-time C++ access rule, not encryption or a complete security boundary.

Both class and struct can use private/public/protected, methods, constructors and inheritance, and either can create automatic or dynamic objects. Their default member and base access differ. A class definition describes a type; creating sword and knife makes separate instances. Ordinary member-function code is not copied into each instance's data storage.

Make the public operation enforce a rule

C++ / WORKED FRAGMENT

```cpp
class Weapon {
    int damage=0;
public:
    void setDamage(int value) {
        if(value>=0) damage=value;
    }
    int getDamage() const { return damage; }
};
// Weapon sword;
// sword.setDamage(10); sword.setDamage(-3);
// sword.getDamage() is still 10; invalid input was rejected.
```

The direct write `sword.damage=-3` is inaccessible from ordinary outside code. Calling setDamage is permitted, but its body decides whether to accept the request. A getter reads without granting direct write access. A setter with no check would still provide no such nonnegative-value policy. The trailing const on getDamage means that reading does not modify the object's ordinary state through this function.

Member access	Own class member code	Qualifying derived member code	Ordinary code in main
public	yes	yes	yes, through an accessible object/ interface
protected	yes	yes, under protected-access rules	no direct access
private	yes	no direct access to the base's private member	no direct access

Friend declarations grant specific additional access. An access label does not remove the private data from a derived object's base subobject. Protected access also has object-expression restrictions; it is not unrestricted access to every arbitrary base object.

11 OOP

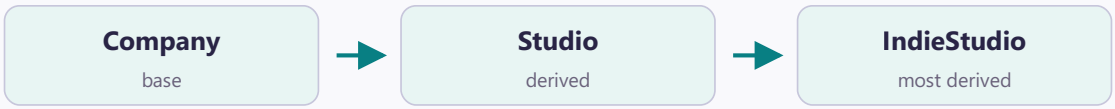
Constructors, destructors and lifetime

Build the foundation first.

Construction, initialization order and destruction

Build, then dismantle

CONSTRUCT THE FOUNDATION FIRST



DESTROY IN REVERSE



This is a lifetime timeline, not a promise about exact byte layout.

PAUSE & PREDICT

Studio derives from Company. Which constructor completes its base initialization first?

A. Studio B. Company C. The pointer variable

B. The base must be initialized before the derived constructor body runs.

Initialization happens before use

A constructor has the class name and no return type. `Weapon(int amount)` is a constructor; `void Weapon(int)` is not. Constructors may be overloaded with different parameter lists. Calling the same constructor with 50 and 100 is two uses of one overload, not two overloads.

A default constructor is one callable without arguments. `Weapon()` is one; `Weapon(int n=0)` can also be one. A parameterized constructor is not automatically accompanied by a zero-argument constructor. When you declare an ordinary constructor, provide a default one explicitly if the desired use needs it.

C++ / WORKED FRAGMENT

```cpp
class Weapon {
    int bullets;
public:
    Weapon() : bullets(0) {}
    explicit Weapon(int amount) : bullets(amount) {}
    int count() const { return bullets; }
};
Weapon empty;
Weapon loaded(50);
```

The colon introduces member initialization, which precedes the constructor body. `bullets(amount)` initializes the member. An assignment inside the body runs after members have already been initialized. This matters for references and const members: they must be initialized, and cannot be left for assignment later. Non-static members initialize in their declaration order, regardless of the order written in the initializer list.

`this` is the pointer to the current object in an ordinary non-static member function. `this->bullets=bullets;` distinguishes a member from a parameter with the same name; `bullets(bullets)` in an initializer list is another valid style. This is not a new object and not a copy of the object. A static member function has no implicit this because no particular instance is required.

Bases first, body later

C++ / WORKED FRAGMENT

```cpp
class Company {
    int id;
public:
    explicit Company(int value) : id(value) {}
};
class Studio : public Company {
public:
    Studio() : Company(5) {}
};
Studio s; // valid: base is explicitly initialized with 5
```

If Studio omits that base initializer and Company has no viable default constructor, construction fails. Calling `Company(5);` as a statement in Studio's constructor body creates a separate temporary; it does not initialize the already-required base subobject.

For Company→Studio→IndieStudio, normal construction finishes Company, then Studio, then IndieStudio. Destruction runs the reverse order. At each level

members are initialized before that level's constructor body. This is a lifetime timeline, not a claim about exact byte order.

Destruction is tied to lifetime

A destructor is written `~Weapon()` with no return type or ordinary parameters. For an automatic local, it normally runs when its scope exits. Two locals created a then b are destroyed b then a on normal scope exit. A dynamic object created with new does not die just because the local pointer goes out of scope; an owning raw-pointer example needs deletion. Deleting invokes the target's destructor and releases the dynamic storage.

An array constructs elements in order and destroys them in reverse on normal destruction. Use delete[] for an array allocated by new[]. A base class intended for deletion through a base pointer normally needs a public virtual destructor. Otherwise deleting a derived object that way has undefined behavior under this course's ordinary case.

Cleanup is one reason copying cannot be specified in isolation. Two copied owning pointers followed by two destructors attempting delete can cause double deletion. Conversely, suppressing deletion merely avoids one symptom while leaking resources. Choose a consistent ownership model.

Read a constructor trace without guessing

1. Confirm that every required constructor is declared, accessible and defined.
2. Identify the actual object and its bases/members.
3. Record base initialization, then members in declaration order, then body.
4. At normal scope exit, reverse the destruction structure.
5. For copy counts, check whether initialization, assignment, move or elision applies before counting.

> Memory anchor: **build the foundation before the room; dismantle the room before the foundation**. Avoid relying on this analogy for virtual calls made during construction/destruction, which have special rules outside the core trace exercise.

A default argument can also make an existing constructor callable without an explicit argument. That is different from asking the compiler to generate an implicit default constructor.

12 OOP

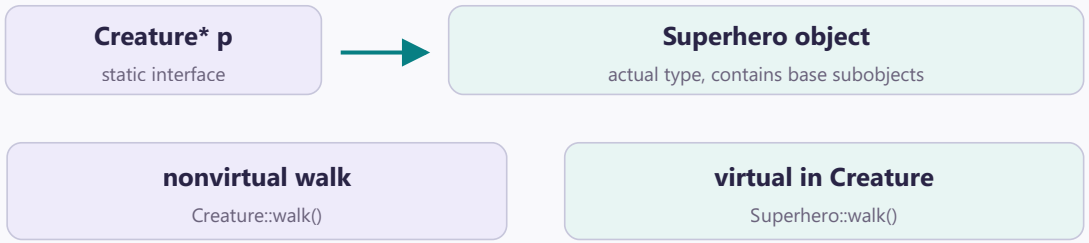
Inheritance and virtual dispatch

Which walk runs?

Static interface, actual type and the final overrider

Choose the implementation

ONE SUPERHERO OBJECT. TWO SEPARATE QUESTIONS.



If Superhero has no override, use the inherited final overrider.

Static type controls the visible interface. A virtual call uses the actual object's final overrider.

PAUSE & PREDICT

Creature* points to Superhero. walk is virtual in Creature and overridden in Superhero. Which walk runs?

A. Creature B. Superhero C. All base and derived walk bodies

B. A matching virtual call uses the actual object's final overrider, not every body in the chain.

Consider Company -> Studio -> IndieStudio and Creature -> Human -> Superhero. With public inheritance, a derived object contains base subobjects. Base private data remains part of the object, though derived member code cannot name it directly. Protected members are accessible within the class and qualifying derived member code, not arbitrary code in main. Public access still requires an available object/interface for non-static members.

`Superhero hero; Creature* p=&hero;` changes the available interface to Creature; hero remains a Superhero. A derived-to-accessible-unambiguous-base conversion is valid. The reverse implicit conversion is invalid. Converting a pointer is not object slicing. Copying a derived object into a separate base object by value does slice away the derived portion of the new copy.

Without virtual in Creature's walk declaration, p->walk() selects Creature::walk according to the static interface. With virtual there, it selects the final overrider for the actual object, Superhero::walk. If Superhero supplies no override but Human does, Human's implementation is used. Declaring walk virtual only in Human does not retroactively make Creature::walk virtual: a Creature* call still uses Creature, while a Human* can dispatch to Superhero. Matching signatures matter; override asks the compiler to check them.

Why use a base pointer at all? An array of Creature pointers can contain Creature, Human and Superhero objects. A loop issues walk() through one interface, and each object responds appropriately. A concrete

motivation is to give every enemy an attack command without writing a separate controller branch for each concrete class.

An abstract base with `virtual void attack()=0;` cannot be instantiated, but can be used as a pointer/reference type. A concrete child must implement all remaining pure virtual operations. C++ has no required interface keyword; a class of pure virtual operations can express an interface. The Weapon/Sword/Gun/Rifle exercise uses attack and refill with a switch menu. Include a virtual destructor when a derived object will be deleted through an owning base pointer.

A complete dispatch example

C++ / WORKED PROGRAM

```cpp
#include <iostream>
struct Creature {
    virtual void walk() const { std::cout << "C"; }
    virtual ~Creature() = default;
};
struct Human : Creature {
    void walk() const override { std::cout << "H"; }
};
struct Superhero : Human {
    void walk() const override { std::cout << "S"; }
};
int main() {
    Creature c;
    Human h;
    Superhero s;
    Creature* group[] = {&c,&h,&s};
    for (Creature* p : group) p->walk(); // CHS
}
```

The array stores three pointers, not three sliced Creature values. The automatic objects outlive the loop. Public struct inheritance is intentional; the same hierarchy with class needs explicit public inheritance and accessible operations.

Removing virtual at Creature and removing now-invalid override specifiers makes calls through Creature* produce CCC. Starting virtual only at Human lets Human* dispatch, but Creature* still selects Creature's nonvirtual declaration. If Superhero omits walk, its inherited final overrider is Human::walk. Data-member access still depends on the static interface; virtual does not expose derived-only members through Creature*.

A pure virtual contract

C++ / WORKED FRAGMENT

```cpp
struct Weapon {
    virtual void attack() = 0;
    virtual void refill() = 0;
    virtual ~Weapon() = default;
};
struct Sword : Weapon {
    void attack() override { /* sword behavior */ }
    void refill() override { /* magical recharge */ }
};
// Weapon w; // ill-formed: abstract
Sword sword;
Weapon* p = &sword;
```

The common contract supports heterogeneous concrete weapons. A derived type implementing only attack can be declared but remains abstract while refill is pure. A pure virtual can have a separate out-of-class definition in advanced uses; =0 defines abstractness, not a universal ban on a body.

One complete object, several legal views

An IndieStudio contains its Studio base subobject, which in turn contains its Company base subobject. The pointer's static type chooses the interface you can name. It does not trim the complete object in memory. Private base data remains part of the base subobject even though derived code cannot directly name it. Actual byte order, padding, and implementation metadata are not determined by a simplified rectangle drawing.

With public, unambiguous inheritance, `Company* p=new IndieStudio;` is a valid derived-to-base conversion. `IndieStudio* p=new Company;` is not its reverse: a standalone Company lacks an IndieStudio's extra state and behavior. Downcasts require their own conditions; do not force an invalid object relationship with a cast. A base pointer also may be adjusted to the appropriate subobject, so avoid promising identical numeric addresses in every hierarchy.

One command can invoke different behavior

Create local Sword, Gun and Rifle instances that publicly derive from an abstract Weapon with virtual attack and refill operations. An array of Weapon pointers can hold their addresses. A loop calling `w->attack()` uses each object's final overrider. Another loop can call refill; a switch menu only chooses which common command to issue. Verify the class behavior before adding the input menu. The array must hold pointers/references to preserve the distinct objects; a base-value container is a different design and may slice or be impossible for an abstract base.

An ordinary walk call invokes one selected body; it does not automatically call the Human and Creature bodies too. A derived method can explicitly call a base implementation when that extra behavior is wanted. Constructor/destructor sequencing is a separate rule from ordinary method dispatch.

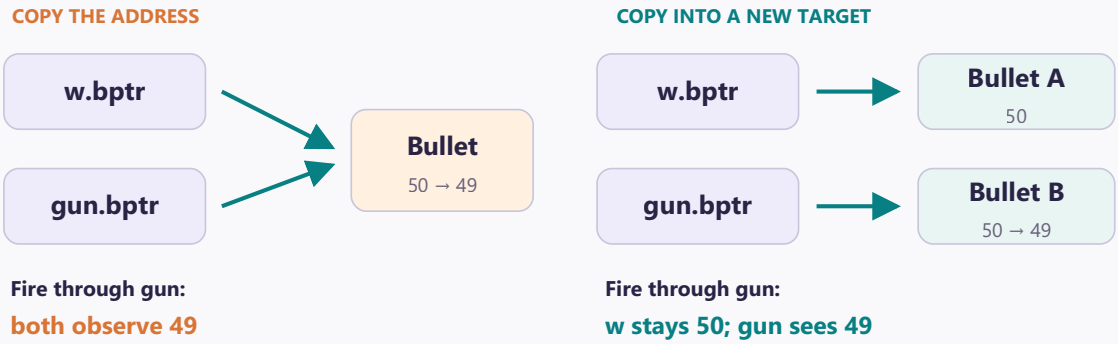
13 OOP

Copying and ownership

Same value. Shared target?

Memberwise copies, independent resources and assignment

Sharing versus independence



Equal values can live in different targets. Shared addresses reach the same target.

PAUSE & PREDICT

A valid pointer to a live Bullet is copied memberwise. What is shared?

A. The entire containing object B. The pointed-to target C. Every int member

B. The pointer members are separate, but contain the same target address. Ordinary int members remain separate.

Copying is memberwise by default. An int member becomes a distinct int value. A raw pointer member becomes a distinct pointer containing the same address. A class member follows its own copy semantics; an object member is not automatically shallow. In the Weapon example, two pointer members share one Bullet, so changing that Bullet is visible through both weapons. A deep copy allocates independent storage and copies the target's value.

`Weapon b(a);` and `Weapon b=a;` create a new b and can invoke a copy constructor such as `Weapon(const Weapon& other)`. In `Weapon b(100); b=a;`, b already exists, so the second line uses copy assignment. Keep these operations distinct. A custom copy constructor alone does not repair copying by assignment, double deletion or a missing destructor. Owning raw pointers normally require a consistent copy constructor, copy assignment and destructor; use value members when pointer ownership is unnecessary.

Returning an object by value does not always execute a copy constructor. Moves and copy elision depend on the code and language standard. A fixed copy count requires those conditions to be specified.

A complete owning-pointer example

This C++17 example demonstrates deep copying while giving assignment and destruction matching behavior. A one-int allocation is a teaching model; an ordinary int value member is the simpler practical design.

C++ / WORKED FRAGMENT

```cpp
class Magazine {
    int* bullets;
public:
    explicit Magazine(int n) : bullets(new int(n)) {}
    Magazine(const Magazine& other)
        : bullets(new int(*other.bullets)) {}
    Magazine& operator=(const Magazine& other) {
        if (this != &other) {
            int* replacement = new int(*other.bullets);
            delete bullets;
            bullets = replacement;
        }
        return *this;
    }
    ~Magazine() { delete bullets; }
    void fire() { --*bullets; }
    int count() const { return *bullets; }
};
// Magazine a(50); Magazine b(a); b.fire();
// a.count()==50; b.count()==49.
// Magazine c(100); c=a; c.fire();
// a.count()==50; c.count()==49.
```

Construction allocates a target; copy construction allocates another initialized from the old value. Copy assignment allocates its replacement before releasing the existing target, preserving the destination if allocation throws. Self-assignment is handled. Each destructor releases its own allocation once. This example omits moves; ordinary copies remain available.

Contrast `int* q=p;`: sharing a target through copied observing pointers can be intentional. Shallow sharing becomes wrong when it contradicts the required ownership/value semantics. Draw shared and independent state before choosing an operation.

Read the pointer member and its target separately

With only int real and int imag members, implicit memberwise copying produces independent int members. Changing the copy's imag does not change the original. Adding `int* x` changes the resource story: copying x's address makes the two pointer members reach the same int. The pointer members remain separate objects, and any ordinary int fields remain independent.

For the Weapon/Bullet example, `bptr=other.bptr` shares the existing Bullet. `bptr=new Bullet(*other.bptr)` constructs a separate Bullet from the existing Bullet's value. First obtain valid independent storage, then copy the value into it; writing `*bptr=*other.bptr` through an uninitialized bptr is invalid. If Bullet itself owns resources, its own copy semantics determine whether that inner copy is deep enough.

Beware an unconditional `new Complex` inside every Complex constructor: each construction starts another before it can finish. A pointer member allows a recursive relationship, but its mere presence does not require endless recursive allocation. A recursive data structure needs a finite construction plan and an empty/terminal link where appropriate.

14 OOP

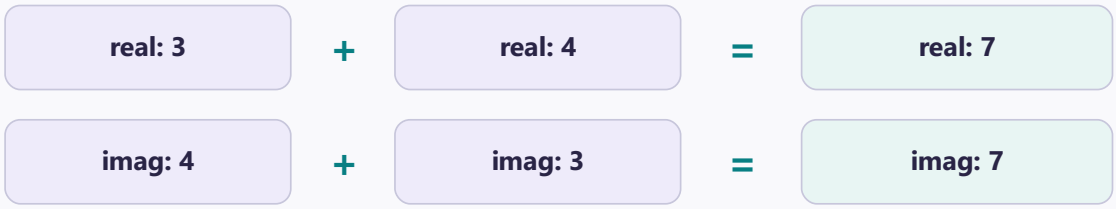
Overloading, overriding and Complex arithmetic

Keep real and imaginary in their lanes.

Overloads, virtual overrides and value arithmetic

Keep the components aligned

ADD CORRESPONDING COMPONENTS



a + b returns a new value. The operands stay unchanged.

Each lane has a defined meaning; C++ does not guess which fields to add.

PAUSE & PREDICT

What distinguishes ordinary overloads?

A. Return type alone B. Parameter names alone C. Distinguishable parameter lists

C. Name changes inside the parameter list do not create new function types; return type alone cannot distinguish the overloads.

Three different decisions

Overloading chooses among functions with the same name and different parameter lists. `int sum(int,int)` and `double sum(double,double)` can coexist. Parameter names do not distinguish overloads; return type alone does not either. Merely calling sum with different values uses the same overload if the argument types select it.

Overriding replaces the implementation of a virtual operation in a derived class. It requires a matching virtual function relationship; unrelated functions named walk are not automatically overrides. Use override on the derived declaration to diagnose a mismatch. Static member functions cannot be virtual.

Operator overloading defines an existing operator's behavior for a user-defined operand type. It does not change precedence, associativity, number of operands, or the built-in meaning of int+int. You cannot invent ** as a new exponentiation operator. Operators such as ::, . and sizeof cannot be overloaded.

Make value arithmetic behave like value arithmetic

C++ / WORKED FRAGMENT

```cpp
struct Complex {
    int real;
    int imag;
    Complex(int r=0, int i=0) : real(r), imag(i) {}

    Complex operator+(const Complex& rhs) const {
        return Complex(real + rhs.real, imag + rhs.imag);
    }
    Complex operator-(const Complex& rhs) const {
        return Complex(real - rhs.real, imag - rhs.imag);
    }
};
// Complex(3,4) + Complex(4,3) has real=7, imag=7.
```

For a member binary operator, the left operand is the current object and rhs is the explicit right parameter. Thus a+b corresponds to a.operator+(b). `const Complex&` avoids an unnecessary argument copy and prevents modifying rhs through that parameter. The trailing const prevents modifying this object's ordinary state through the member function.

The result is a separate value. Returning a reference to a local temporary would leave a dangling reference after the function returns. Returning a new-allocated pointer would change the result type and burden the caller with ownership. A value return expresses the arithmetic result naturally; elision/moves can avoid copies.

Ordinary + should not silently change a. `+=` is a mutating operation: it can update real and imag and return `Complex&` to the same object. The arithmetic example returns a separate value without modifying either operand.

Increment is a design choice for Complex

There is no universally required mathematical meaning for incrementing a complex number. For this exercise, increment both components. State that choice before asking for an output.

C++ / WORKED FRAGMENT

```cpp
// Members inside Complex, under the exercise's chosen semantics:
Complex& operator++() {       // prefix
    ++real;
    ++imag;
    return *this;
}
Complex operator++(int) {     // postfix: dummy int distinguishes it
    Complex old = *this;
    ++(*this);
    return old;
}
```

For prefix, the returned reference denotes the updated object. For postfix, preserve the old value, then update the object, and return the old value. The dummy int is a signature convention; writing x++ does not mean the student passes a meaningful integer argument. Decrement can mirror this chosen exercise behavior.

Do not predict ++ behavior for a class whose overloads are not shown. Unlike the built-in int case, a class operator's body determines what it actually does. Likewise std::cout cannot print an arbitrary Complex merely because + exists: provide a print function or an appropriate output operator.

Printing negative components

If imag=-2, blindly printing real<<" + "<<imag<<"i" produces a readable but awkward '+ -2i'. Formatting is separate from arithmetic. A conditional format can choose a minus and the magnitude. The representation stores signed components; a printed plus sign does not change them.

> Memory anchor: **overload: choose a signature; override: choose an implementation; operator overload: give a symbol a class meaning**.

Components stay in their own lanes

Represent a complex value as real + imag*i. (3+4i) +(4+3i) gives 7+7i; subtraction gives -1+1i. The output string is formatting, separate from the stored numbers. Stream insertion `std::cout<<real<<" + "<<imag<<"i"` writes the pieces in sequence; it is not a general C++ rule that an int plus an empty string becomes a string. `operator+` is not an output operator and does not automatically make `std::cout<<value` valid.

When no appropriate overload exists, adding two arbitrary class objects is a compilation error, not a request for the compiler to guess how to add fields. In the increment exercise, both components change because that policy is explicitly implemented, not because C++ knows a universal mathematical ++ rule for complex numbers.

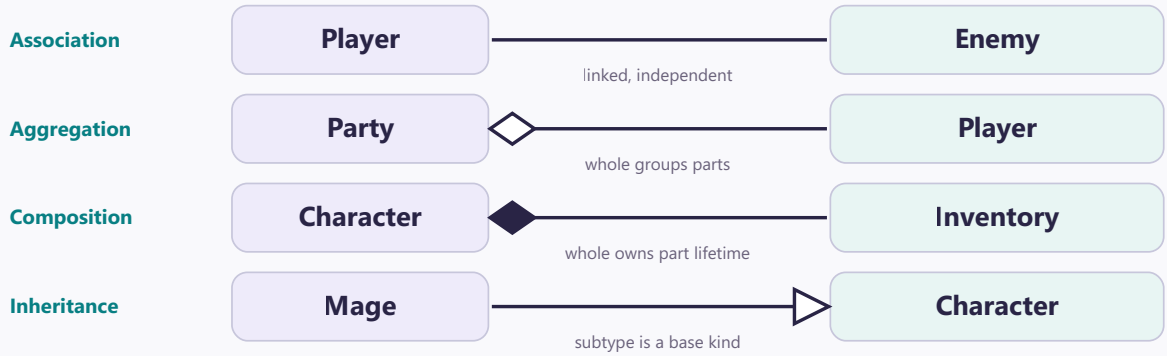
---

15 **Design** UML and object relationships

What does this arrow promise?

Association, aggregation, composition and inheritance

Read the relationship symbols



The diamond sits at the whole; the triangle points at the general base.

PAUSE & PREDICT

Where is the filled composition diamond drawn?

A. At the whole B. At the part C. Always at the left page edge

A. The diamond identifies the owning whole, whichever side of the diagram it occupies.

Start from the modeled responsibility

A class box has three compartments: its name, attributes, and operations. Use + public, - private, # protected. `+attack(): void` describes an operation, `-health: int` an attribute. Parameters and return types make the intended interface clearer. Static members can be underlined in UML; an abstract class/operation can be italicized or marked with an explicit abstract annotation.

To reverse-engineer a game model, choose an existing game, identify its main classes and useful methods, then justify the relations. Include meaningful operations as well as class names. Use a modest, defensible model rather than pretending a diagram reconstructs the actual game's source code.

Association: an object knows or uses another

An association is a structural relationship between instances. Draw a solid line. A Player may target an Enemy. Specifying 'one target at a time, target exists independently' makes the model concrete. The Player does not own the Enemy's lifetime merely because it stores a pointer to it.

The mnemonic 'uses' needs one distinction. In fuller UML, a transient parameter-only use is often a dependency, drawn dashed, whereas a maintained link is an association. In the four-way comparison below, association represents the maintained link. Association does not mean a destructor must delete the linked object.

Aggregation: a whole groups independent parts

Shared aggregation uses a hollow diamond at the whole end. A Party groups Players that remain alive if the Party is dissolved; a player can leave or be reassigned under this model. The important assumption is independent lifetime, not just whether the source code contains a pointer.

Car and Wheel can fit a detachable-part model, but the same nouns do not force that relationship in every system. UML shared aggregation deliberately leaves some semantics to the model. State those semantics instead of inferring ownership from a noun.

Composition: a whole owns its parts

Composition uses a filled diamond at the whole end. A Character owns an Inventory object as part of that character's lifetime in our model. Destroying the character destroys that inventory object. The Inventory may itself own items, depending on the model; composition is not a promise that every referenced item must die.

A value member `Inventory inventory;` is a simple C++ expression of such ownership. An owning smart pointer can also implement composition. A raw pointer can represent either ownership or observation, so its syntax alone cannot prove aggregation or composition. A part can be moved between wholes if the model permits; composition forbids simultaneously composing the same part into two wholes.

Inheritance: the subtype can serve as the base

Generalization uses a solid line ending in a hollow triangle pointing at the base. Mage is a Character; Mage should work where the public Character interface is expected. This is different from Character has an Inventory. Reusing fields alone is not enough reason to assert an is-a relationship.

A base-pointer view is the code counterpart of this arrow. It does not mean the base holds every future derived member. Private/protected/public visibility is separate from the relationship kind.

Compare by asking what happens to the other object

Relationship	Key question	Symbol	Our game example
Association	Does one maintain a non-owning link to another?	solid line	Player targets Enemy
Shared aggregation	Does a whole group independently existing parts?	hollow diamond at whole	Party groups Players
Composition	Does a whole own its part's lifetime?	filled diamond at whole	Character owns Inventory
Inheritance	Can this subtype fulfill that base interface?	hollow triangle at base	Mage is Character

Multiplicity labels describe how many instances may participate: 1 exactly one, 0..1 optional one, * zero or more, 1..* one or more. A Party with 1..4 Players is a modeling choice; a label does not allocate four C++ objects by itself.

Diamond recognition

If Student and Faculty each inherit Person, and TA inherits both, a nonvirtual inheritance model gives TA two Person base subobjects. Naming a common Person member can become ambiguous. This is the diamond inheritance pattern. Multiple inheritance is supported by C++, not prohibited. Virtual inheritance can share a virtual base, but its construction rules are an optional extension here. Do not confuse virtual inheritance with virtual member-function dispatch.

A small game model

Draw Character with +attack(), +takeDamage(amount:int), -health:int. Derive Mage and Knight. Give Character a composed Inventory and a non-owning target association. Let Party aggregate Characters under the independent-lifetime assumption. Write one sentence justifying each arrow. Then ask whether a common Character* loop can command each derived attack: that ties UML to behavior instead of decoration.

> Memory anchor: **line: linked; hollow diamond: grouped; filled diamond: owned; triangle: is-a**. Always locate the whole or the base before choosing arrow direction.

Build a diagram that explains behavior

For an existing game, identify major responsibilities, give each class a useful name, then add the operations that make it participate in gameplay. Include attributes where they clarify the model; a class need not be filled with invented fields just to occupy a compartment. The class names, methods, and justified relationships are the core of the diagram. A clear attack/refill interface explains more than a large collection of empty boxes.

Locate the whole before drawing either diamond, and locate the general base before drawing the inheritance triangle. Labels such as owns, groups and targets should state the intended lifetime policy. Reverse engineering here means proposing a defensible model from observable game behavior; it does not reveal the actual private implementation.

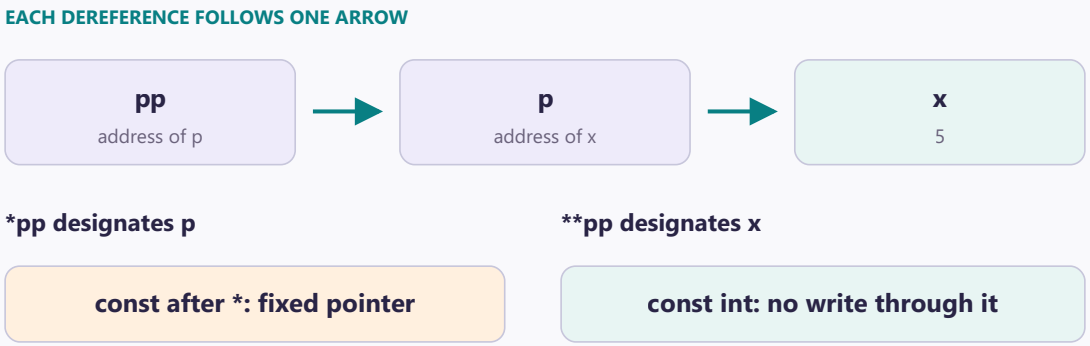
16 **Extensions**

pointer chains, const and void*

Follow one arrow at a time.

Pointer chains, const restrictions and generic object addresses

Follow one arrow at a time



Follow the declared types and live objects at each step; a cast does not create a new reality.

PAUSE & PREDICT

What can int* const p=&x still permit for a mutable live x?

A. Reseating p B. Writing through *p C. Both restrictions disappear

B. Const after the star fixes the pointer object. The pointed-to int is still mutable.

Add one arrow at a time

C++ / WORKED FRAGMENT

```cpp
int x = 5;
int* p = &x;
int** pp = &p;
```

`pp` has type int**; `*pp` is an int* lvalue; `**pp` is an int lvalue. `**pp=8` changes x. Given `int y=9; *pp=&y;`, p now points to y while pp still points to p. That changes the intermediate pointer, not x's stored value. Do not treat pp+1 as advancing through x values: it would advance through an array of int* objects, if pp belonged to such an array.

This enables a function to reseat a caller's pointer: `void redirect(int** out,int* target){*out=target;}` called as redirect(&p,&y). A reference-to-pointer parameter `int*&` is another way. Both forms allow the function to change the caller's pointer object.

Read const near the thing it restricts

Declaration	Change pointer target?	Write target through p?
`int* p`	yes	yes, if target is mutable/live
`const int* p` / `int const* p`	yes	no
`int* const p=&x`	no	yes, if x is mutable
`const int* const p=&x`	no	no

Const after the star restricts the pointer object. Const attached to int restricts the pointed-to int access. `const int* p=&x; x=9;` is valid if x was originally a mutable int: p still observes 9, though `*p=9` is forbidden. The restriction is not a freeze on every possible access path. You cannot safely cast away const and modify an object originally defined const.

A const pointer is still a pointer object; the array mnemonic does not make an array that pointer type. A reference is also not literally a const pointer, although both cannot be reseated in normal use.

A generic address is not a generic value

C++ / WORKED FRAGMENT

```cpp
int x = 3;
void* generic = &x;
int* typed = static_cast<int*>(generic);
// *typed is 3
```

Void has no object size or member interface, so `*generic` is ill-formed and arithmetic on void* is not standard C++. Convert back to a suitable actual object type before accessing. A cast does not create an object of the requested type or validate a wrong guess. The fact that an int and a float can have the same size does not authorize reading one through the other's pointer type.

This is about object pointers. A guarantee for arbitrary function-pointer conversion to void* does not follow. For function callbacks, preserve the declared compatible function-pointer type.

Arithmetic rules worth distinguishing

Within one array, p+n and p-n move by elements, and p-q yields an element distance if both positions belong to the same array or its one-past position. `p-=q` is ill-formed: compound subtraction on a pointer requires an integer displacement, not another pointer. `p+q`, pointer multiplication and pointer division are ill-formed.

`nullptr` is a null pointer constant with its own type and is clearer in C++ than old NULL macros. Null is different from a pointer one-past an array and different from a pointer to destroyed storage. All three have different reasons not to be dereferenced.

> Memory anchor: **each star follows one arrow; const says which operation is closed; a cast changes the view, not the stored reality**.

A fixed route and a read-only route differ

For `int* const p=&x`, the route is fixed but a mutable x may change through *p. For `const int* p=&x`, the route may move but writing through it is restricted. The familiar array-name cue resembles the first restriction only; it does not make the array a pointer object. An ordinary int array can still have mutable elements. Null pointer, one-past pointer and dangling pointer are distinct states, with different reasons that dereferencing them is invalid.

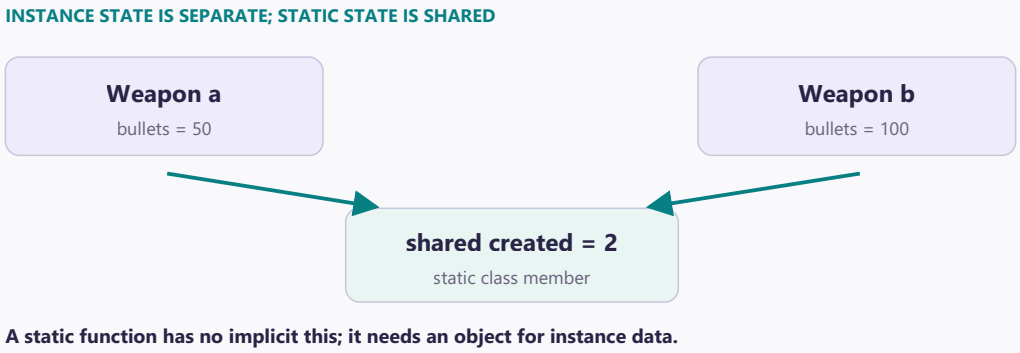
---

17 **Extensions** instance and static members

Which state belongs to everyone?

Instance members, class state and static functions

Per object versus shared



This counter records the shown construction path; it is not automatically a live-object count.

PAUSE & PREDICT

A static member function receives no explicit instance. Does it have an implicit this?

A. Yes B. No C. Only when called through an object

B. A static member function has no implicit selected object. Calling syntax does not add this.

What belongs to one object?

A non-static data member is part of each object's state. If two Weapon instances have bullets=50 and bullets=100, changing one need not change the other. An ordinary non-static member function operates on a selected object and has an implicit this pointer. `w.count()` selects w; `p->count()` selects the pointed-to object.

A static data member belongs to the class's shared state, rather than being a separate value in every instance. A static member function has no implicit selected object. It can access static members directly, but needs an explicit object/pointer/reference to access non-static state.

C++ / WORKED FRAGMENT

```cpp
class Weapon {
    int bullets;
    inline static int created = 0; // C++17 inline variable
public:
    explicit Weapon(int n) : bullets(n) { ++created; }
    int count() const { return bullets; }
    static int totalCreated() { return created; }
};
Weapon a(50), b(100);
// a.count()==50, b.count()==100, Weapon::totalCreated()==2.
```

This counter counts calls to the shown constructor, not all possible copies or currently alive objects. To count live objects correctly, define what copying/moving means and adjust every relevant constructor/destructor. Avoid a visual that labels a simplistic counter 'live' when it only increments.

Before C++17, a common pattern declares `static int created;` in the class and defines `int Weapon::created=0;` in one source file. A declaration alone is not normally a storage definition. Using the C++17 inline pattern above keeps this lesson's single-file example self-contained.

Static functions can be invoked with class scope such as Weapon::totalCreated(). An ordinary non-static count needs an object. `Weapon::count()` with no object is ill-formed. Calling a static function through an object is permitted but class scope expresses the shared nature more clearly.

Three meanings people accidentally mix

Phrase	Scope/access	Lifetime or relationship
local static variable	function/block name	persists across calls after initialization
static data member	class access rules	one shared member, not one per instance
namespace-scope static	translation-unit internal linkage	different topic from local scope

A static local inside main remains a name in main's block, not a global name. Do not infer visibility from longevity.

Encapsulation is a decision about permitted operations

Do not automatically generate a setter for every field. A generated ID may be readable but immutable through the public interface; a health value might change only through takeDamage and heal rather than a direct setHealth. A setter can validate, clamp or reject input. Without such policy, private-plus-trivial-getter/setter can merely relocate unrestricted mutation.

Abstraction asks what a caller needs to know: attack() may hide ammunition arithmetic and animation scheduling. Encapsulation gives the type control over how its state may change. They support each other but answer different questions.

> Memory anchor: **instance: this object's data; static: the class's shared data; public API: the operations callers are allowed to request**.

Shared state is not shared instance data

Two Weapon objects each own their ordinary bullets member even when a shared static counter belongs to the class. Calling a static function through one instance does not give that function an implicit this pointer. A local static inside an ordinary member function is another separate case: its name is local to that function and its object persists across calls, including calls made on different instances.